

Trivy

Submitted by	Vraj Soni, Aditya singh, Anamay Narkar
Roll number	16010122309, 16010123025, 16010123040
Institute	KJ Somaiya College of Engineering
Department	Computer Engineering
Subject	Network Information Security
Date	March 31, 2026
Video Presentation	https://drive.google.com/file/d/1WrIzaG75DAR1KZMzdAWwW5xR56OR52n3/view?usp=drive_link
PPT URL	https://docs.google.com/presentation/d/1zBzmInSwR6beNP_pDXEdo3Xccvee8okA/edit?usp=sharing&rtpof=true&sd=true
Github REPO URL	https://github.com/AnamayNarkar/NIS-IA

Abstract:

This report presents a comprehensive study and practical demonstration of Trivy, an open-source vulnerability and misconfiguration scanner developed by Aqua Security. The study covers Trivy's capabilities for scanning Docker images, filesystems, and application dependencies for known CVEs, exposed secrets, and security misconfigurations. A sample Go web application connecting to MongoDB with a hardcoded URI was developed and containerized using an intentionally outdated base OS image to simulate real-world vulnerabilities. Practical experiments were conducted to demonstrate OS-level, application-level, and secret-detection scanning. The report also addresses Trivy's limitations in secret detection and demonstrates how custom YAML configuration policies can extend its capabilities to catch non-standard secrets such as MongoDB connection strings

Objectives:

- To understand the purpose and working of Trivy as a container and filesystem security scanning tool
- To set up a vulnerable sample Go application with a hardcoded MongoDB URI and outdated Docker base image
- To demonstrate OS-level vulnerability detection by scanning a Docker image built on an old base OS
- To identify application-level vulnerabilities in Go standard library dependencies
- To demonstrate secret detection using Trivy's built-in scanner and compare results with a custom regex-based exploit script
- To highlight the limitations of Trivy's default secret detection, specifically its failure to detect MongoDB URIs
- To write and apply a custom Trivy YAML configuration file to extend secret detection rules
- To analyze and interpret scan results in the context of container and application security

Introduction:

Trivy (pronounced 'tri-vee') is a comprehensive and easy-to-use open-source security scanner developed by Aqua Security. It is designed to find vulnerabilities, misconfigurations, secrets, and software bill of materials (SBOM) in a wide range of targets — including container images, filesystems, Git repositories, Kubernetes clusters, and cloud infrastructure.

Unlike traditional scanners that focus solely on OS packages, Trivy takes a layered approach: it scans both the operating system and application dependencies (such as Go modules, npm packages, Python pip packages, etc.) within a single scan pass. It maps findings to known CVEs from multiple vulnerability databases including the National Vulnerability Database (NVD), GitHub Advisory Database, and OS-specific advisories.

In this assessment, Trivy was installed via the Arch Linux pacman package manager and used to scan a custom Go web application containerized with Docker. The application was intentionally built with an outdated base OS image and a hardcoded MongoDB connection URI to simulate a realistic misconfigured production environment. All scanning was performed on equipment owned by the team members within a controlled academic environment.

Features of the Tool:

1. Multi-Target Scanning

1. Docker image scanning (trivy image) for OS packages and app dependencies
2. Filesystem scanning (trivy fs) for source code, configs, and lock files
3. Git repository scanning (trivy repo) for remote codebases
4. Kubernetes cluster scanning (trivy k8s) for workload misconfigurations

2. Vulnerability Detection

5. Scans OS packages (apt, yum, apk) against CVE databases
6. Scans language-specific dependencies: Go modules, npm, pip, Maven, Cargo, etc.
7. Displays severity levels: CRITICAL, HIGH, MEDIUM, LOW, UNKNOWN
8. Maps findings to CVE IDs with CVSS scores and fix versions where available

3. Secret Detection

9. Built-in detection for popular secret formats: AWS credentials, GitHub tokens, private keys, etc.
10. Configurable via custom YAML rules using regex patterns
11. Can be applied during both image and filesystem scans

4. Misconfiguration Detection

12. Scans Dockerfiles, Kubernetes YAML, Terraform, and CloudFormation for misconfigs
13. Uses built-in policies from the Trivy policy bundle (OPA Rego rules)

5. SBOM Generation

14. Generates a Software Bill of Materials in CycloneDX or SPDX formats
15. Useful for supply chain security and compliance reporting

6. Output Formats

16. Human-readable table output (default), JSON, SARIF, CycloneDX, and template-based outputs
17. Easily integrated into CI/CD pipelines with exit code control (--exit-code 1)

7. Installation Flexibility

- 18.** Available via package managers: pacman (Arch), apt (Debian/Ubuntu), brew (macOS)
- 19.** Available as a Docker image, binary download, and GitHub Actions integration

Installation & Setup:

Trivy is available in the Arch Linux community repository and can be installed directly using the pacman package manager:

```
[anamay@archhouse build]$ sudo pacman -S trivy
[sudo] password for anamay:
warning: trivy-0.69.3-1 is up to date -- reinstalling
resolving dependencies...
looking for conflicting packages...

Packages (1) trivy-0.69.3-1
Total Installed Size: 237.79 MiB
Net Upgrade Size:      0.00 MiB

:: Proceed with installation? [Y/n] Y
(1/1) checking keys in keyring
(1/1) checking package integrity
(1/1) loading package files
(1/1) checking for file conflicts
(1/1) checking available disk space
:: Processing package changes...
(1/1) reinstalling trivy
:: Running post-transaction hooks...
(1/1) Arming ConditionNeedsUpdate...
[anamay@archhouse build]$ trivy --version
Version: 0.69.3
Vulnerability DB:
  Version: 2
  UpdatedAt: 2026-03-31 06:57:03.955552098 +0000 UTC
  NextUpdate: 2026-04-01 06:57:03.955551767 +0000 UTC
  DownloadedAt: 2026-03-31 11:13:08.91752845 +0000 UTC
[anamay@archhouse build]$
```

installed via pacman - package manager

Experimentation & Results:

We have created a sample app which is a basic http server in golang.

The repo for the app is : <https://github.com/AnamayNarkar/NIS-IA>

The app is a basic http server which connects to mongo db as the persistence layer and exposes two api endpoints

POST /users - to create new users in the database

```
router.POST("/users", func(c *gin.Context) {
    var user User
    if err := c.ShouldBindJSON(&user); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
        return
    }

    insertCtx, insertCancel := context.WithTimeout(context.Background(), 5*time.Second)
    defer insertCancel()

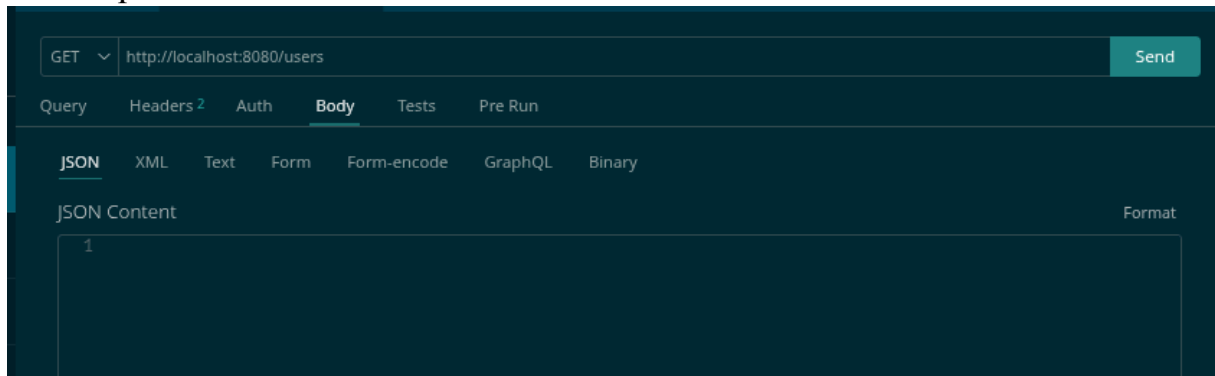
    result, err := collection.InsertOne(insertCtx, bson.M{
        "username": user.Username,
        "email":    user.Email,
        "created":  time.Now().UTC(),
    })
    if err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error": err.Error()})
        return
    }

    c.JSON(http.StatusCreated, gin.H{"id": result.InsertedID})
})
```

GET /users - to get all the users from the database

Calling the endpoints:

Get Request



Get Response

```
Status: 200 OK   Size: 295 Bytes   Time: 7 ms

Response  Headers 4  Cookies  Results  Docs

1  [
2  {
3    "username": "alice",
4    "email": "alice@example.com"
5  },
6  {
7    "username": "alice",
8    "email": "alice@example.com"
9  },
10 {
11   "username": "alice",
12   "email": "alice@example.com"
13 },
14 {
15   "username": "alice",
16   "email": "alice@example.com"
17 },
18 {
19   "username": "alice",
20   "email": "alice@example.com"
21 },
22 {
23   "username": "alice",
24   "email": "alice@example.com"
25 }
26 ]
```

Post request

```
POST  http://localhost:8080/users  Send

Query  Headers 2  Auth  Body 1  Tests  Pre Run

JSON  XML  Text  Form  Form-encode  GraphQL  Binary

JSON Content  Format

1  {
2    "username": "alice",
3    "email": "alice@example.com"
4  }
```

Post Response

Status: 201 Created Size: 33 Bytes Time: 25 ms				
Response	Headers 4	Cookies	Results	Docs
1 {				
2 "id": "69cd49d8ac2c140bc01ff867"				
3 }				

The app is dockerized and run alongside a docker image of mongodb using docker compose

```
[anamay@archhouse app]$ docker compose up --build
WARN[0000] /home/anamay/Programming/college/NIS-IA-Trivy/app/docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid potential confusion
WARN[0000] Docker Compose requires buildx plugin to be installed
Sending build context to Docker daemon  2.817MB
Step 1/13 : FROM golang:1.21-bullseye AS builder
--> 5e3b31e004ad
Step 2/13 : WORKDIR /app
```

For the Docker base image in this demonstration, we intentionally utilized an outdated Debian 8 image to ensure the presence of unpatched vulnerabilities for scanning purposes.

```
FROM golang:1.21-bullseye AS builder

WORKDIR /app

ENV GO111MODULE=on

COPY go.mod ./
COPY vendor ./vendor
COPY . ./
RUN go build -mod=vendor -o server ./main.go

FROM debian:9

WORKDIR /app
COPY --from=builder /app/server /app/server

EXPOSE 8080
CMD ["/app/server"]
```

While the application functions correctly, it contains a severe vulnerability: the MongoDB connection URI is hardcoded. Furthermore, due to the outdated base image, the OS contains multiple CRITICAL and HIGH vulnerabilities

```
const mongoURI = "mongodb://mongo:27017"
```

and because of the old image there are many CRITICAL and HIGH vulnerabilities in the OS.

If a malicious actor gains access to the Docker image or the codebase, they can easily extract these hardcoded credentials using basic regex scripts, posing a severe security threat

```
[anamay@archhouse exploit]$ ./exploit-bin
[hardcoded-key] ../app/main.go:14
Possible hardcoded secret/key value
const mongoURI = "mongodb://mongo:27017"

[anamay@archhouse exploit]$ █
```

Similarly, a hacker could exploit the OS level vulnerabilities to cause damage to organizations.

Trivy can help tremendously here.

If we run trivy against the docker image we can get a very well documented list of the vulnerabilities and which versions fix it

```
[anamay@archhouse exploit]$ trivy image b0681d3cef4f
2026-04-01T22:15:26+05:30 INFO [vulndb] Need to update DB
2026-04-01T22:15:26+05:30 INFO [vulndb] Downloading vulnerability DB...
2026-04-01T22:15:26+05:30 INFO [vulndb] Downloading artifact... repo="min
89.19 MiB / 89.19 MiB [-----]
2026-04-01T22:15:54+05:30 INFO [vulndb] Artifact successfully downloaded
2026-04-01T22:15:54+05:30 INFO [vuln] Vulnerability scanning is enabled
2026-04-01T22:15:54+05:30 INFO [secret] Secret scanning is enabled
2026-04-01T22:15:54+05:30 INFO [secret] If your scanning is slow, please try '--
2026-04-01T22:15:54+05:30 INFO [secret] Please see https://trivy.dev/docs/v0.69/
2026-04-01T22:15:58+05:30 INFO Detected OS family="debian" version="9.13"
2026-04-01T22:15:58+05:30 INFO [debian] Detecting vulnerabilities... os_version
2026-04-01T22:15:58+05:30 INFO Number of language-specific files num=1
2026-04-01T22:15:58+05:30 INFO [gobinary] Detecting vulnerabilities...
2026-04-01T22:15:58+05:30 WARN Using severities from other vendors for some vuln
2026-04-01T22:15:58+05:30 WARN This OS version is no longer supported by the dis
2026-04-01T22:15:58+05:30 WARN The vulnerability detection may be insufficient b
```

OS Level vulnerabilities

b0681d3cef4f (debian 9.13)

Total: 71 (UNKNOWN: 0, LOW: 36, MEDIUM: 5, HIGH: 28, CRITICAL: 2)

Library	Vulnerability	Severity	Status	Installed Version	Fixed Version	Title
bsdutils	CVE-2016-2779	HIGH	affected	1:2.29.2-1+deb9u1		util-linux: runuser tty hijack via TIOCSTI ioctl https://avd.aquasec.com/nvd/cve-2016-2779
	CVE-2021-37600	LOW				util-linux: integer overflow can lead to buffer overflow in get_sem_elements() in sys-utils/lpcutils.c... https://avd.aquasec.com/nvd/cve-2021-37600
coreutils	CVE-2016-2781		will_not_fix	8.26-3		coreutils: Non-privileged session can escape to the parent session in chroot https://avd.aquasec.com/nvd/cve-2016-2781
e2fslibs	CVE-2022-1304	HIGH	affected	1.43.4-2+deb9u2		e2fsprogs: out-of-bounds read/write via crafted filesystem https://avd.aquasec.com/nvd/cve-2022-1304
e2fsprogs						
gcc-6-base	CVE-2018-12886		will_not_fix	6.3.0-18+deb9u1		gcc: spilling of stack protection address in cfgexpand.c and function.c leads to... https://avd.aquasec.com/nvd/cve-2018-12886
gpgv	CVE-2018-1000858		affected	2.1.18-8+deb9u4		gnupg2: Cross site request forgery in dimngr resulting in an information disclosure... https://avd.aquasec.com/nvd/cve-2018-1000858
	CVE-2018-9234	LOW				GnuPG: Unenforced configuration allows for apparently valid certifications actually signed by signing... https://avd.aquasec.com/nvd/cve-2018-9234
	CVE-2019-14855					gnupg2: OpenPGP Key Certification Forgeries with SHA-1 https://avd.aquasec.com/nvd/cve-2019-14855
libblkid1	CVE-2016-2779	HIGH		2.29.2-1+deb9u1		util-linux: runuser tty hijack via TIOCSTI ioctl https://avd.aquasec.com/nvd/cve-2016-2779
	CVE-2021-37600	LOW				util-linux: integer overflow can lead to buffer overflow in get_sem_elements() in sys-utils/lpcutils.c... https://avd.aquasec.com/nvd/cve-2021-37600
libbz2-1.0	CVE-2019-12900	CRITICAL		1.0.6-8.1		bzip2: bzip2: Data integrity error when decompressing (with data integrity tests fail).... https://avd.aquasec.com/nvd/cve-2019-12900
libcomerr2	CVE-2022-1304	HIGH		1.43.4-2+deb9u2		e2fsprogs: out-of-bounds read/write via crafted filesystem https://avd.aquasec.com/nvd/cve-2022-1304
libdb5.3	CVE-2019-8457	CRITICAL	will_not_fix	5.3.28-12+deb9u1		sqlite: heap out-of-bound read in function rtreenode() https://avd.aquasec.com/nvd/cve-2019-8457
libelf1	CVE-2018-16403	LOW	affected	0.168-1+deb9u1		elfutils: Heap-based buffer over-read in libdw/dwarf_getabbrev.c and libdw/dwarf_hasattr.c causes crash https://avd.aquasec.com/nvd/cve-2018-16403
	CVE-2019-7149					elfutils: heap-based buffer over-read in read_srclines in dwarf_getsrclines.c in libdw https://avd.aquasec.com/nvd/cve-2019-7149

Example Vulnerability: For instance, the libbz2-1.0 library has a CRITICAL vulnerability (CVE-2019-12900) causing data integrity errors during decompression. The installed version is 1.0.6-8.1.

Identifying Patches & Key Observations: Typically, you check the "Fixed Version" column for the patched package release. However, in this scan, the "Fixed Version" column is completely empty, and statuses show "affected" or "will_not_fix." This indicates that no patched versions are currently available in your OS repositories. To secure your system, you must wait for vendor patches, upgrade to a newer OS distribution, or implement compensating security controls.

Go std library vulnerabilities

github.com/gin-gonic/gin	CVE-2020-28483	HIGH	fixed	v1.6.3	1.7.7	gin: HTTP response splitting https://avd.aquasec.com/nvd/cve-2020-28483
	CVE-2023-26125	MEDIUM			1.9.0	golang-github-gin-gonic-gin: Improper Input Validation https://avd.aquasec.com/nvd/cve-2023-26125
	CVE-2023-29401				1.9.1	golang-github-gin-gonic-gin: Gin Web Framework does not properly sanitize Filename parameter of Context.FileAttachment... https://avd.aquasec.com/nvd/cve-2023-29401
golang.org/x/crypto	CVE-2024-45337	CRITICAL		v0.26.0	0.31.0	golang.org/x/crypto/ssh: Misuse of ServerConfig.PublicKeyCallback may cause authorization bypass in golang.org/x/crypto https://avd.aquasec.com/nvd/cve-2024-45337
	CVE-2025-22869	HIGH			0.35.0	golang.org/x/crypto/ssh: Denial of Service in the Key Exchange of golang.org/x/crypto/ssh https://avd.aquasec.com/nvd/cve-2025-22869
	CVE-2025-47914	MEDIUM			0.45.0	golang.org/x/crypto/ssh/agent: SSH Agent servers: Denial of Service due to malformed messages https://avd.aquasec.com/nvd/cve-2025-47914
	CVE-2025-58181					golang.org/x/crypto/ssh: golang.org/x/crypto/ssh: Denial of Service via unbounded memory consumption in GSSAPI authentication... https://avd.aquasec.com/nvd/cve-2025-58181
stdlib	CVE-2025-68121	CRITICAL		v1.21.13	1.24.13, 1.25.7, 1.26.0-rc.3	crypto/tls: Unexpected session resumption in crypto/tls https://avd.aquasec.com/nvd/cve-2025-68121
	CVE-2024-34156	HIGH			1.22.7, 1.23.1	encoding/gob: gob: Calling Decoder.Decode on a message which contains deeply nested structures... https://avd.aquasec.com/nvd/cve-2024-34156
	CVE-2025-47907				1.23.12, 1.24.6	database/sql: Postgres Scan Race Condition https://avd.aquasec.com/nvd/cve-2025-47907
	CVE-2025-58183				1.24.8, 1.25.2	golang: archive/tar: Unbounded allocation when parsing GNU sparse map https://avd.aquasec.com/nvd/cve-2025-58183
	CVE-2025-61726				1.24.12, 1.25.6	golang: net/url: Memory exhaustion in query parameter parsing in net/url https://avd.aquasec.com/nvd/cve-2025-61726
	CVE-2025-61728					golang: archive/zip: Excessive CPU consumption when building archive index in archive/zip https://avd.aquasec.com/nvd/cve-2025-61728
	CVE-2025-61729				1.24.11, 1.25.5	crypto/x509: golang: Denial of Service due to excessive resource consumption via crafted... https://avd.aquasec.com/nvd/cve-2025-61729
	CVE-2026-25679				1.25.8, 1.26.1	net/url: Incorrect parsing of IPv6 host literals in net/url https://avd.aquasec.com/nvd/cve-2026-25679
	CVE-2024-34155	MEDIUM			1.22.7, 1.23.1	go/parser: golang: Calling any of the Parse functions containing deeply nested literals... https://avd.aquasec.com/nvd/cve-2024-34155
	CVE-2024-34158					go/build/constraint: golang: Calling Parse on a "// +build" build tag line with... https://avd.aquasec.com/nvd/cve-2024-34158
	CVE-2024-45336				1.22.11, 1.23.5, 1.24.0-rc.2	golang: net/http: net/http: sensitive headers incorrectly sent after cross-domain redirect https://avd.aquasec.com/nvd/cve-2024-45336

This table unequivocally presents the comprehensive results of a security vulnerability scan, specifically from Trivy, run against our Go application's image. It meticulously categorizes identified vulnerabilities across three primary dependencies: the `github.com/gin-gonic/gin` web framework, the `golang.org/x/crypto` cryptographic library, and the Go standard library (`stdlib`). For every single vulnerability discovered, the report explicitly lists the CVE identifier, its severity rating ranging from MEDIUM up to CRITICAL, the version currently installed in your image, and the target version where the patch is applied. Furthermore, it provides a highly specific explanation of each flaw - such as HTTP response splitting in Gin, SSH authorization bypasses in x/crypto, or memory exhaustion in the standard library - alongside direct reference links to the Aqua Security vulnerability database (avd.aquasec.com) for remediation.

Report Summary

Report Summary			
Target	Type	Vulnerabilities	Secrets
b0681d3cef4f (debian 9.13)	debian	71	-
app/server	gobinary	38	-

Legend:

- '-': Not scanned
- '0': Clean (no security findings detected)

As we can see trivy is really great for analyzing vulnerable versions of software and get suggestions for patches. But there is no mention of the Mongo hardcoded URI

This is because trivy does do secret scanning but its for more popular key formats like AWS credentials or github tokens. We could add this functionality by adding a manual config for regex scanning.

```
trivy-secret.yaml x
trivy-secret.yaml
1 rules:
2   - id: hardcoded-mongodb-uri
3     category: custom
4     title: Hardcoded MongoDB URI
5     severity: HIGH
6     regex: "(?i)(mongo(uri)?|mongodb[_-]?uri|connection[_-]?string)\\s*[:=]\\s*[\"']mongodb://[^\"]+\\s*\""
7     keywords:
8       - mongo
9       - mongodb
10
```

If we pass this config as a scanner config in a cli argument, we get the following

output :

```
[anamay@archhouse NIS-IA-Trivy]$ trivy fs --scanners secret --secret-config trivy-secret.yaml --skip-dirs vendor app/
2026-04-01T22:32:25+05:30      INFO      [secret] Secret scanning is enabled
2026-04-01T22:32:25+05:30      INFO      [secret] Please see https://trivy.dev/docs/v0.69/guide/scanner/secret#recommendation for faster secret detection
2026-04-01T22:32:25+05:30      INFO      [secret] Loading the config file for secret scanning... config_path="trivy-secret.yaml"
2026-04-01T22:32:26+05:30      INFO      Number of language-specific files      num=1

Report Summary



| Target  | Type  | Secrets |
|---------|-------|---------|
| go.mod  | gomod | -       |
| main.go | text  | 1       |



Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)

main.go (secrets)
Total: 1 (UNKNOWN: 0, LOW: 0, MEDIUM: 0, HIGH: 1, CRITICAL: 0)
HIGH: custom (hardcoded-mongodb-uri)

Hardcoded MongoDB URI
main.go:14 (offset: 210 bytes)
12 )
13
14 [ const "*****"
15
```

As we can see, we are now getting a custom error of mongo db being hardcoded as a HIGH level security vulnerability.

We will now also showcase SBOMs using trivy,

To demonstrate Trivy's Software Bill of Materials (SBOM) functionality, we ran

```
[anamay@archhouse NIS-IA-Trivy]$ trivy image --format cyclonedx -o sbom.json b0681d3cef4f
```

.This successfully generated a complete inventory of all packages and dependencies in the container, which is a critical feature for compliance and supply chain auditing.

```

{
  "$schema": "http://cyclonedx.org/schema/bom-1.6.schema.json",
  "bomFormat": "CycloneDX",
  "specVersion": "1.6",
  "serialNumber": "urn:uuid:fbec6967-8317-42b1-be68-b067531bf65a",
  "version": 1,
  "metadata": {
    "timestamp": "2026-04-01T17:20:53+00:00",
    "tools": {
      "components": [
        {
          "type": "application",
          "manufacturer": {
            "name": "Aqua Security Software Ltd."
          },
          "group": "aquasecurity",
          "name": "trivy",
          "version": "0.69.3"
        }
      ]
    },
    "component": {
      "bom-ref": "4b48c19a-f2ba-4611-bca9-cabaf0bee6dd",
      "type": "container",
      "name": "b0681d3cef4f",
      "properties": [
        {
          "name": "aquasecurity:trivy:DiffID",
          "value": "sha256:47046c6b29c38a95c4fa495a9f2a17a9296696f2f17648d4b7fa3f72491cacc6"
        }
      ]
    }
  }
}

```

This is a small part of a large output, the full output is on

<https://github.com/AnamayNarkar/NIS-IA/blob/main/sbom.json>

It is essential for supply chain security. If a major zero-day vulnerability is discovered tomorrow in a specific library (like the infamous Log4j flaw), security teams don't have to spend hours rescanning their entire infrastructure. They can simply search their generated SBOMs to instantly see if they are using that vulnerable "ingredient."

Limitations:

1. Additional Overhead for Custom Scanning: While Trivy is highly extensible, configuring and maintaining custom secret scanners (such as the regex rules required to catch the MongoDB URI) introduces additional administrative overhead for development and security teams.
2. Inability to Detect Logical Issues: Trivy is a static vulnerability and misconfiguration scanner. It cannot identify logical flaws, business logic errors, or architectural vulnerabilities within the application runtime that a Dynamic Application Security Testing (DAST) tool or manual penetration test would

uncover.

3. False Positives with OS Packages: Like many static scanners, Trivy can occasionally report false positives for OS-level vulnerabilities. This often occurs when OS vendors backport security patches into older package versions without incrementing the primary version number, causing the scanner to flag a patched package as vulnerable.

4. Supply Chain Risks of Security Tooling: As a highly privileged tool operating within CI/CD pipelines, scanners like Trivy are high-value targets for upstream supply chain attacks. Relying on mutable version tags (e.g., `:latest`) for the scanner's image or GitHub Actions can expose the environment to compromised binaries or credential theft, requiring strict mitigation like SHA256 digest pinning.

Analysis of Outcome:

The practical execution of Trivy against the deliberately vulnerable target yielded several critical insights into container security. First, the OS-level scan effectively demonstrated the severe risks of using outdated base images. By identifying unpatched CVEs in the Debian 8 environment (such as the CRITICAL flaw in libbz2-1.0), Trivy provided immediate visibility into the underlying infrastructure's attack surface. Furthermore, the tool successfully mapped application-level vulnerabilities within the Go dependencies (e.g., Gin framework and crypto libraries) to specific CVEs, proving its utility in securing the software supply chain.

However, the most significant outcome of this experiment was the analysis of Trivy's secret detection capabilities. The initial scans failed to flag the hardcoded MongoDB URI, highlighting a critical blind spot: default secret scanners are heavily optimized for standard cloud provider tokens (like AWS or GitHub) and often miss application-specific credentials. By successfully writing and applying a custom YAML regex policy, we demonstrated that Trivy's true value lies in its extensibility. The outcome proves that while Trivy is an exceptionally fast and comprehensive baseline scanner, organizations cannot rely solely on its out-of-the-box configurations. It must be actively tuned and customized with localized rules to ensure robust, environment-specific security posture.

Conclusion:

Trivy proves to be a highly effective and versatile layer of defense for securing containerized applications, as demonstrated by its ability to rapidly identify critical OS-level vulnerabilities in the outdated Debian base image and vulnerable dependencies in the Go environment. Its ease of integration and unified scanning approach make it an essential baseline tool for DevSecOps pipelines. However, this assessment also highlights that Trivy is not a comprehensive standalone solution. Its default secret scanning can miss non-standard, critical exposures like hardcoded database URIs, and its static nature means it cannot detect logical application flaws. To maximize its effectiveness, organizations must extend Trivy with custom policies tailored to their specific tech stack, combine it with dynamic testing methods, and strictly secure the pipeline infrastructure running the scans.